Nova: Agent Specification (V4) — Compliance & Gap Summary

Audit Date: October 10, 2026 | Workspace: `nova-agent` | Reference Document: *Nova: Agent Specification (V4)*

Executive Verdict: The current `nova-agent` repository is **partially aligned in core domain math and UI**, but **architecturally non-compliant** with *Nova: Agent Specification (V4)*. The codebase implements an earlier **standalone stateful V1/V2.0-Agentic architecture** in which Nova owns the SQLite database, categorizes raw expenses locally, executes all deterministic math internally, exposes direct state-mutating REST routes, and negotiates directly with Atlas. By contrast, V4 requires Nova to be a **stateless LLM specialist** with **no write tools**, operating in a strict **star topology** under John, reading categorized snapshots only through **Sentinel's 6 functions**, and communicating via `TaskPacket@1` and `SpecialistResponse@1`.

1. Section-by-Section Compliance Matrix

V4 Specification Area	Status	What IS in the Codebase	What IS NOT / Divergent
Header & Changes from V3 (Page 1)	**NOT MET**	No `RECATEGORIZE_TRANSACTION` proposal is emitted by Nova; protected categories exist in persona preferences.	Missing Tally `categorization.updated` event trigger, taxonomy `category_id` hierarchy & migrations, `spent_settled` / `spent_provisional` / `tier_is_provisional`, `CAP_ON_DEPRECATED_CATEGORY`, `data_quality`, and `MISCATEGORIZATION_SUSPECTED` hint.
1–3. Overview, Persona & Ecosystem Map (Pages 2–3)	**PARTIAL / DIVERGENT**	Active real-time Budget Guardian persona; vigilant, fact-based, non-punitive tone; prepares coaching brief for John (`AgentNegotiator.js`).	Violates **star topology** (Nova directly messages Atlas in `negotiateWithAtlas`); Nova computes math directly instead of calling Sentinel; Nova owns state & write routes instead of delegating writes to an Action Service.
4. Real-Time Architecture & Thresholds (Page 3)	**PARTIAL / MISMATCH**	Calendar month budget period (`YYYY-MM`); deterministic threshold check runs in <2ms without waiting for LLM completion.	Warning threshold is hardcoded at **85%** instead of **80%** (and Exceeded at `≥100%` instead of `>100%`); triggered by raw expense POST instead of Tally batch/ correction; no tier-change deduplication.
5. Data Contracts (`TaskPacket@1` & `SpecialistResponse@1`, Pages 4–5)	**MISSING**	Returns custom JSON telemetry & OODA cycle object containing executive summary, velocity, anomalies, and rebalance plan.	No `TaskPacket@1` input (takes `userId` and reads raw ledger rows); no `SpecialistResponse@1` output shape; missing `status` enum, `findings[]` with `{{placeholders}}`, `boundary_alerts[]`, `constraints[]`, `proposed_actions[]`, `peer_hints[]`, `confidence`, and `tool_result_ids[]`.

6. Sentinel Functions & Cap Rules (Pages 5–6)	**PARTIAL**	Core formulas for utilization, velocity/burn rate, anomalies, and zero-sum rebalance exist in `src/engine/`.	Not structured as the 6 Sentinel functions; missing `sentinel_get_ledger_summary` (aggregates only) and `sentinel_validate_cap_change`; missing all 5 cap rule codes (`CAP_NEGATIVE`, `CATEGORY_UNKNOWN`, `PROTECTED_CATEGORY`, `CAPS_EXCEED_SPENDABLE`, `CAP_ON_DEPRECATED_CATEGORY`).
7–8. Interaction Patterns & Lifecycle (Pages 6–7)	**PARTIAL**	Implements breach + rebalance calculation (Pattern A) and subscription price hike detection (Pattern B).	OODA loop runs all tools unconditionally instead of conditional execution by `task_type`; Pattern C negotiates directly with Atlas instead of returning `GOAL_FUNDING_PRESSURE` hint; Pattern D (category correction updating both categories) is missing.
9 & 14. Guardrails & Platform Rules (Pages 7–8, 10–11)	**PARTIAL**	Protected categories excluded as rebalance donors; deterministic alert generation precedes LLM narrative; non-punitive tone.	Write endpoints exist without confirmation tokens; no provisional breach labeling; no conservative daily envelope fallback when confidence != high; no prompt-injection sanitization on merchant strings.
10–13. Routing, Scenarios, Errors & Eval (Pages 8–10)	**MOSTLY MISSING**	11 automated tests in `tests/novaEngine.test.js` covering baseline math, anomalies, velocity, and rebalance.	Missing Section 12 error handlers (Sentinel timeout fallback to aged snapshot, stale snapshot re-run, `needs_input` on ambiguous category, `NEEDS_REVIEW`, `UNCATEGORIZED`, duplicate event suppression).

2. Detailed Breakdown: What IS There (Implemented in Codebase)

- **Category Utilization & Net Surplus Math (`src/engine/NovaGuardianEngine.js`):**
  - Aggregates month-to-date category spend for the calendar month (`YYYY-MM`).
  - Computes utilization percentage (`used_pct`), remaining allowance (`monthly_cap - spent`), and category status (`healthy`, `warning`, `exceeded`).
  - Tracks uncapped categories separately and computes net monthly cash flow surplus (`monthly_income - fixed_housing_expense - totalVariableSpent`) and savings rate.
- **Velocity Projection & Daily Safe Envelope (`src/engine/tools/VelocityPredictor.js`):**
  - Computes `daily_burn_rate`, `projected_month_end_spend`, `projected_overrun`, `days_remaining`, and `days_until_breach`.
  - Calculates `safe_daily_spend_envelope` per category and globally across capped categories.
- **Anomaly Detection Suite (`src/engine/tools/AnomalyDetector.js`):**
  - *Duplicate Swipes (`POTENTIAL_DUPLICATE`):* Detects identical amount and category charges within 30 minutes.
  - *Single-Transaction Spikes (`CATEGORY_WHALE_SPIKE`):* Flags single transactions consuming ≥45% of a monthly category cap.
  - *Subscription Price Creep (`SUBSCRIPTION_PRICE_CREEP`):* Compares recurring charges against `merchant_profiles.typical_amount` and flags increases >15%.
- **Zero-Sum Rebalance Simulation & Protected Categories (`src/engine/tools/RebalanceEngine.js`):**
  - Finds deficit categories (`remaining < 0`) and donor categories (`remaining > 500`).
  - Reads `priority_categories` (e.g., Housing, Healthcare, Bills & Utilities) from `user_personas` and excludes them as donors.
  - Generates zero-sum transfer moves with updated donor and recipient caps.

- **Real-Time Deterministic Priority & Narrative Synthesis (`src/engine/NovaAgentBrain.js`):**
  - Threshold alerts are generated deterministically by `NovaGuardianEngine` without waiting for LLM completion.
  - Synthesizes executive summaries, root-cause diagnostics, and John coaching briefs via Gemini 1.5 Flash with a deterministic local fallback.
- **No Recategorization Proposal:** Nova never proposes a `RECATEGORIZE_TRANSACTION` action.
- **Nova Budget Workspace UI (`src/client/NovaGuardianView.jsx`):** React dashboard rendering category caps, progress meters, status badges, safe daily spending envelope pill, rebalance card, and transaction feed.

3. Detailed Breakdown: What IS NOT There (Missing or Divergent from V4)

3.1 Architectural, Topology & Permission Gaps

- **Star Topology Violation:** In V4, Nova talks *only* to John and Sentinel, passing cross-agent signals as `peer_hints[]` (`GOAL_FUNDING_PRESSURE`, `RECURRING_ANOMALY`, `MISCATEGORIZATION_SUSPECTED`). In the codebase (`AgentNegotiator.js`), Nova directly negotiates with Atlas (`negotiateWithAtlas`) and persists transcripts in `agent_negotiations`.
- **Stateless Specialist & No-Write Rule Violation:** V4 prohibits write tools on Nova and mandates that only the backend **Action Service** mutates state after user confirmation token verification. The codebase gives Nova direct SQLite write endpoints (`PUT /api/budget/:category`, `DELETE /api/budget/:category`, `POST /api/nova/telemetry/rebalance`, `POST /api/transactions`) with no confirmation tokens.
- **Sentinel & Tally Separation:** V4 separates transaction categorization into **Tally** and mathematical calculations into **Sentinel** (accessed via 6 specific functions pinned to a `snapshot_id`). Currently, Nova runs keyword categorization locally (`Categorizer.js`) and queries raw SQLite tables directly.
- **Warning Tier Threshold Mismatch (85% vs. 80%):** V4 defines `Healthy < 80%`, `Warning 80%–100%`, and `Exceeded > 100%` as Sentinel configuration. The codebase hardcodes Warning at `85%–99%` and Exceeded at `≥100%` in both `NovaGuardianEngine.js` and `NovaGuardianView.jsx`.

3.2 Missing Data Contracts & Conditional Execution Lifecycle

- **Missing `TaskPacket@1` Input (Section 5.1):** No support for `TaskPacket@1` fields (`task_type`, `snapshot_id`, `user_message`/`event`, `ui_context`, `data_quality`, `preferences`, `rolling_summary`, `recent_turns`, `peer_findings`, `allowed_tools`, `budget`). Violates the rule: *"No `user_id`, no raw ledger."*
- **Missing Conditional Execution by `task_type` (Section 8):** Does not implement conditional tool ordering for the 6 task types:
  - `live_status` → utilization, velocity
  - `breach_event` → utilization, velocity, rebalance simulation, cap validation
  - `anomaly_event` / `explain_anomaly` → anomalies, ledger summary
  - `propose_rebalance` → rebalance simulation, cap validation
  - `adjust_cap_request` → cap validation, utilization
- **Missing `SpecialistResponse@1` Output (Section 5.3):** Does not return `status` (`ok | partial | needs_input | rejected | error`), `findings[]` with bound `{{placeholders}}`, `category_status_summary`, `boundary_alerts[]`, `daily_spending_envelope`, `constraints[]`, `proposed_actions[]`, `peer_hints[]`, `confidence`, or `tool_result_ids[]`.

3.3 Missing V4 Feature Upgrades (Changes from V3)

- **Tally `categorization.updated` Event & Category Correction Recomputation:** No event consumer for `categorization.updated`. Moving a transaction between categories (Pattern D, e.g., Dining to Shopping) is not implemented and does not recompute both the old and new category tiers to resolve/create alerts in the same cycle.
- **Taxonomy `category_id`, Subcategory Hierarchy & Migration Map:** Caps are keyed to plain-text category names rather than a taxonomy `category_id`. Parent category caps do not cover subcategories, and there is no migration map for renamed, merged, or split categories.
- **Provisional vs. Settled Spend (`spent_settled`, `spent_provisional`, `tier_is_provisional`):** Utilization does not split settled spend from `NEEDS_REVIEW` provisional spend. Provisional breaches are not labeled `provisional` and do not link to the review list.
- **Uncategorized Bucket & Data Quality (`UNCATEGORIZED`, `data_quality`):** Unmatched transactions fall back to

`Miscellaneous` instead of `UNCATEGORIZED`; uncategorized totals are not displayed alongside live status when material, and `import_in_progress` / stale data caveats are missing.

- **Cap Validation Function (`sentinel_validate_cap_change`) & 5 Cap Rules:** Missing validation for `CAP_NEGATIVE`, `CATEGORY_UNKNOWN`, `PROTECTED_CATEGORY` (blocking cap reduction on protected categories unless explicitly requested), `CAPS_EXCEED_SPENDABLE` (warning when sum of caps exceeds spendable income), and `CAP_ON_DEPRECATED_CATEGORY` (warning + replacement suggestion).

3.4 Missing Guardrails, Security & Error Handling (Sections 9, 12, 13, 14)

- **Alert Hygiene & Deduplication:** No stored alert tier state per category; repeated queries or duplicate events regenerate alerts without checking for a tier change, and dismissed alerts do not stay suppressed until a worse tier.
- **Margin of Safety (Conservative Envelope):** Missing `confidence` and `conservative` daily envelope calculation; cannot present the conservative envelope as the headline value when confidence is not `high`.
- **Grounded Number Placeholders:** Gemini prompt path outputs raw numbers in JSON text rather than binding placeholders (`{{placeholder}}`) to Sentinel `tool_result_id`s.
- **Untrusted Text & Prompt Injection Protection:** Merchant names and remarks are not sanitized as untrusted data fields, and there is no gateway allowlist enforcement.
- **Section 12 Error Handling Table:** Missing handlers for Sentinel error/timeout (fallback to last good snapshot labeled with age), stale snapshot re-run, and ambiguous category requests returning `needs_input` with candidate categories.

4. V4 Acceptance Criteria Scorecard (Section 13)

#	Criterion	Status	Notes
1	Tier change detected and alert stored within agreed cycle target (<60s) in 95% of test events.	**PARTIAL**	Math runs in <2ms, but alerts are not stored/ deduplicated on tier changes.
2	Alert is visible without any LLM call having completed.	**MET**	`NovaGuardianEngine` creates alerts deterministically without waiting for LLM.
3	Zero ungrounded numbers in Nova's eval set.	**PARTIAL**	Local synthesizer uses deterministic numbers, but lacks `{{placeholder}}` binding to `tool_result_id`.
4	Nova's allowlist has no write tools; a write attempt fails at the gateway.	**NOT MET**	Nova has direct DB write methods and mutating REST endpoints.
5	No proposal ever includes a protected category as a donor.	**MET**	`RebalanceEngine.js` excludes `priority_categories` from donor list.
6	No cap change or rebalance is executed without a confirmation token.	**NOT MET**	Cap updates and rebalances execute directly on REST call without confirmation tokens.
7	Duplicate events create no duplicate alerts.	**NOT MET**	No event deduplication or persisted tier-change state machine.
8	With non-high confidence, the conservative envelope is the headline in 100% of cases.	**NOT MET**	No `confidence` metric or conservative envelope calculation exists.
9	Injection strings in merchant names produce no calls outside allowlist and no state change.	**NOT MET**	No merchant string sanitization or tool allowlist gateway.

10	Correction moving a transaction between categories updates both categories' tiers and resolves/creates alerts in same cycle.	**NOT MET**	No transaction correction handler or dual-category tier recomputation.
11	Tier that depends only on provisional amounts is labeled provisional in 100% of cases.	**NOT MET**	No provisional vs. settled amount tracking (`tier_is_provisional`).
12	Nova's proposals contain no recategorization action.	**MET**	Nova never proposes `RECATEGORIZE_TRANSACTION`.